

Relatório - Sistema de Autocomplete de Jogos com Trie

Alunos: Elian Miguel Ferreira Custódio, Luis Felipe Veloso Canha

1. Representação da Trie

O alfabeto escolhido possui tamanho 36, definido pela constante `ALPHABET_SIZE = 36`, cobrindo as 26 letras minúsculas do alfabeto (a-z) e os 10 dígitos (0–9).

A função `toSearchKey(std::string)` padroniza um prefixo antes de qualquer inserção ou consulta, removendo espaços e aplicando caixa baixa (*lowercase*). Letras maiúsculas (ASCII 65–90) são convertidas para minúsculas, somando 32 ao valor ASCII. Os outros caracteres são mantidos inalterados.

A função `getIndex(char c)` mapeia cada caractere para a posição correspondente no array `children` de cada nó:

- **Letras minúsculas (a-z):** índice = $c - 'a'$, retornando valores de 0 a 25;
- **Dígitos (0-9):** índice = $26 + (c - '0')$, retornando valores de 26 a 35;
- **Demais caracteres:** retorna -1 , indicando que não deve ser inserido na Trie.

2. Funcionamento dos métodos

2.1 insert

Percorre a Trie caractere a caractere usando a padronização feita pelo `toSearchKey()`. Para cada caractere, calcula o índice via `getIndex()` e cria um novo `TrieNode` filho caso ele não exista. No fim, marca o nó como `isEndOfTitle = true` e guarda o ponteiro para o objeto `Game`.

2.2 contains

Percorre a Trie seguindo os filhos correspondentes a cada caractere da chave normalizada. Se em algum ponto o filho necessário for nulo, retorna *false* imediatamente. Ao passar por todos os caracteres, retorna *true* somente se o nó atual tiver `isEndOfTitle = true`, garantindo que o título exista de forma completa, e não apenas como prefixo de outro.

2.3 autocomplete

Primeiro navega até o nó correspondente ao prefixo normalizado, seguindo a mesma lógica do `contains`. Se o prefixo não existir na *Trie*, retorna um vetor vazio. Se existir, chama o método auxiliar `search()` a partir desse nó, que realiza uma busca por toda a subárvore coletando os jogos cujos títulos completam o prefixo. Os resultados são ordenados por popularidade e em caso de empate, por ordem alfabética, usando o algoritmo *Insertion Sort*. Os *k* primeiros resultados da ordenação são retornados.

3. Custo computacional

insert: Um nó é criado para cada caractere da chave, totalizando um percurso de L passos.

- Custo de tempo: $O(L)$
- Custo de espaço: $O(L)$

contains: Percorre no máximo L arestas da raiz até o nó terminal, e não cria outras estruturas.

- Custo de tempo: $O(L)$
- Custo de espaço: $O(1)$

autocomplete: p passos para percorrer até o prefixo, e mais s operações para varrer a subárvore. Para ordenar, são necessárias r^2 operações no pior caso.

- Custo de tempo: $O(p + s + r^2)$
- Custo de espaço: $O(r)$

4. Comparação e uso de memória

Uma solução ingênua armazenaria todos os jogos N em um array e, a cada consulta de *contains* ou *autocomplete*, percorreria todos os elementos comparando strings, custando ao todo $O(NL)$ por operação. Com a *Trie*, *contains* e *insert* custam $O(L)$ e *autocomplete* custa $O(p + s + r^2)$, tornando as buscas independentes de N .

Com relação à memória, cada *TrieNode* aloca um array fixo de 36 ponteiros (*children*), independentemente de quantos filhos realmente existem. Para um alfabeto de tamanho N jogos com títulos de comprimento médio L , o custo assintótico de memória é $O(NL)$ tanto para a *Trie* quanto para um array de strings, mas a *Trie* tem uma constante multiplicativa maior na prática, pois cada nó reserva 36 ponteiros independentemente de quantos filhos realmente usa, enquanto um array de strings aloca exatamente o espaço necessário por caractere. A principal vantagem estrutural da *Trie* é que títulos com prefixos comuns compartilham os mesmos nós. Por exemplo, "*farcry*", "*farcry2*", "*farcry3*" e "*farcry6*" compartilham os 6 primeiros nós. Quanto maior a sobreposição de prefixos, menor o número total de nós criados e, portanto, menor o consumo de memória, podendo ficar bem abaixo do pior caso $O(NL)$.

O tamanho do alfabeto A afeta diretamente o espaço de cada nó: cada *TrieNode* reserva A ponteiros, mesmo que a maioria seja nula. Com $A = 36$ e ponteiros de 8 bytes, cada nó ocupa pelo menos 288 bytes só para o array de filhos, além dos campos *isEndOfTitle* e *game*. Um alfabeto menor reduziria essa sobrecarga de memória por nó. Um maior aumentaria o custo proporcionalmente a A , tornando a *Trie* pior em memória em relação a outras estruturas.

